

SMART E-COMMERCE PLATFORM

Customer Experience & Insights Module

User-side Refund & Return Request Flow

Submitted by: Ram Kumar

Project: Smart E-Commerce Platform

Module: Customer Experience & Insights

Technology: Python, FastAPI, MySQL, SQLAlchemy, React/Next.js

Feature: User-side Refund & Return Request Flow

1. INTRODUCTION

The Customer Experience & Insights Module extends the Smart E-Commerce Platform by providing users with a structured **Return Request workflow**.

The module allows customers to request a return for eligible delivered orders. The request is validated based on the order status and return period and is stored in the database for further processing.

The system allows customers to:

- View their orders.
- Identify eligible delivered orders.
- Request a return.
- Provide a return reason.
- Add an optional comment.
- Track the return request status.

2. RETURN REQUEST FEATURE

A **Request Return** option has been integrated into the User Orders page.

Customers can request a return only when the order satisfies the required conditions.

Return Eligibility

A return request is allowed when:

- Order status is **Delivered**.
- The order is within the **7-day return window**.
- A return request has not already been submitted for the order.

This validation is handled by the FastAPI backend.

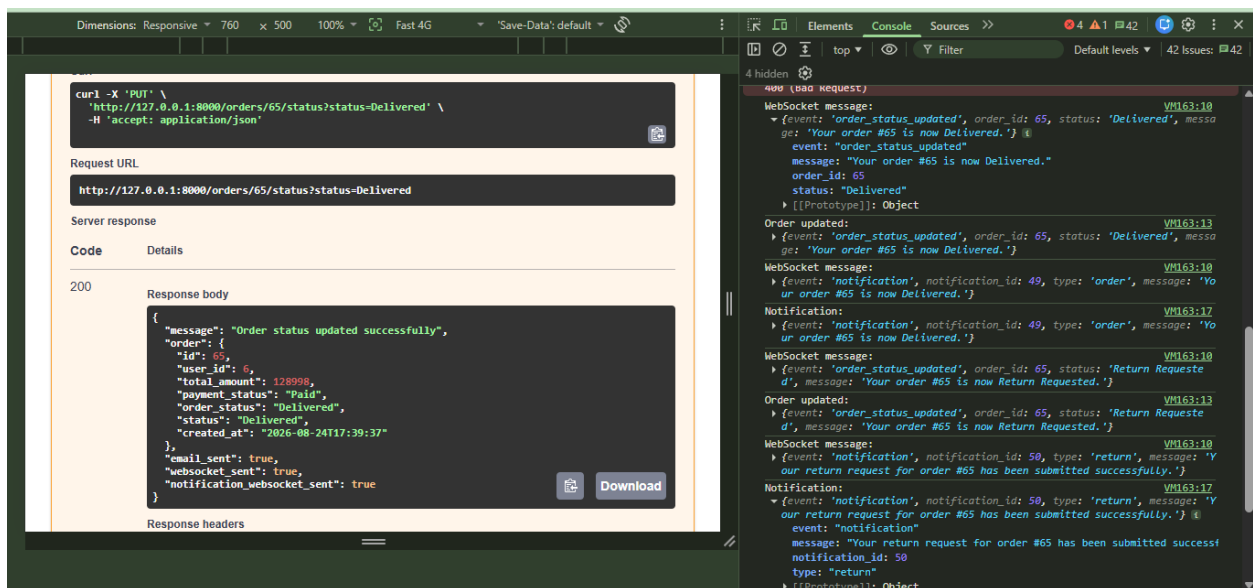


Fig for Delivered

```

{
  "reason": "damage",
  "comment": "not working"
},
}

```

Request URL

http://127.0.0.1:8000/orders/85/return

Server response

Code	Details
200	Response body <pre> { "message": "Return request submitted successfully", "return_request": { "id": 3, "order_id": 85, "user_id": 5, "reason": "damage", "comment": "not working", "status": "Pending", "created_at": "2026-08-31T06:58:33" }, "order_status": "Return Requested", "email_sent": true, "websocket_sent": false, "notification_websocket_sent": false } </pre> Response headers <pre> access-control-allow-credentials: true content-length: 310 content-type: application/json date: Mon, 31 Aug 2026 06:58:33 GMT server: uvicorn </pre>

Responses

Code	Description	Links
------	-------------	-------

Fig for Return Requested

```

Curl
curl -X 'PUT' \
  'http://127.0.0.1:8000/orders/returns/3/approve' \
  -H 'accept: application/json'

```

Request URL

http://127.0.0.1:8000/orders/returns/3/approve

Server response

Code	Details
200	Response body <pre> { "message": "Return approved", "return_request_id": 3, "status": "Approved", "order_status": "Returned" } </pre> Response headers <pre> access-control-allow-credentials: true content-length: 97 content-type: application/json date: Mon, 31 Aug 2026 07:02:19 GMT server: uvicorn </pre>

Responses

Code	Description	Links
200	Successful Response	No links

Fig for Return Request Approved

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/orders/49/return' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "reason": "damage",
    "comment": "not working"
  }'
```

Request URL

`http://127.0.0.1:8000/orders/49/return`

Server response

Code	Details
400	Error: Bad Request

Response body

```
{
  "detail": "Return window expired. Returns are allowed within 7 days."
}
```

Response headers

```
access-control-allow-credentials: true
content-length: 70
content-type: application/json
date: Mon, 31 Aug 2026 07:05:29 GMT
server: uvicorn
```

Responses

Code	Description
------	-------------

Links

Fig for Return Window Expired,Return Allowed Within 7 days

127.0.0.1:8000/docs/#/Orders/reject_return_orders_returns_return_id_reject_put

Dimensions: Responsive x 760 x 500 100% Fast 4G 'Save-Data': default

Curl

```
curl -X 'PUT' \
  'http://127.0.0.1:8000/orders/returns/6/reject' \
  -H 'accept: application/json'
```

Request URL

`http://127.0.0.1:8000/orders/returns/6/reject`

Server response

Code	Details
200	

Response body

```
{
  "message": "Return rejected successfully",
  "return_request_id": 6,
  "status": "rejected",
  "order_status": "Delivered",
  "email_sent": true,
  "websocket_sent": true,
  "notification_websocket_sent": true
}
```

Response headers

```
access-control-allow-credentials: true
content-length: 186
content-type: application/json
date: Mon, 31 Aug 2026 09:14:35 GMT
```

Console

```
<(error) => {
  console.error("WebSocket error:", error);
}

WebSocket connected VM481:4
WebSocket message: VM481:10
  {event: 'connected', user_id: 6, message: 'WebSocket connected successfully'}
WebSocket message: VM462:10
  {event: 'order_status_updated', order_id: 87, status: 'Delivered', message: 'Your order #87 is now Delivered.'}
Order updated: VM462:13
  {event: 'order_status_updated', order_id: 87, status: 'Delivered', message: 'Your order #87 is now Delivered.'}
WebSocket message: VM481:10
  {event: 'order_status_updated', order_id: 87, status: 'Delivered', message: 'Your order #87 is now Delivered.'}
Order updated: VM481:13
  {event: 'order_status_updated', order_id: 87, status: 'Delivered', message: 'Your order #87 is now Delivered.'}
WebSocket message: VM462:10
  {event: 'notification', notification_id: 74, type: 'return', message: 'Your return request for order #87 has been rejected.'}
Notification: VM462:17
  {event: 'notification', notification_id: 74, type: 'return', message: 'Your return request for order #87 has been rejected.'}
WebSocket message: VM481:10
  {event: 'notification', notification_id: 74, type: 'return', message: 'Your return request for order #87 has been rejected.'}
Notification: VM481:17
  {event: 'notification', notification_id: 74, type: 'return', message: 'Your return request for order #87 has been rejected.'}
```

Fig For Reject Request

3. USER RETURN REQUEST WORKFLOW

The implemented workflow is:

- **User Orders**
- **Select Delivered Order**
- **Click Request Return**
- **Enter Return Reason**
- **Enter Optional Comment**
- **Submit Return Request**
- **Backend Validation**
- **ReturnRequest Created**
- **Order Status → Return Requested**

The user receives confirmation after successfully submitting the return request.

Orders		^
POST	/orders/create Create Order	▼
GET	/orders/ Get Orders	▼
PUT	/orders/{order_id}/pay Payment Success	▼
PUT	/orders/{order_id}/status Update Order Status	▼
GET	/orders/analytics/top-selling Top Selling Products	▼
GET	/orders/returns Get All Return Requests	▼
GET	/orders/returns/{return_id} Get Return Request	▼
POST	/orders/{order_id}/return Request Return	▼
PUT	/orders/returns/{return_id}/approve Approve Return	▼
PUT	/orders/returns/{return_id}/reject Reject Return	▼
GET	/orders/{order_id} Get Order	▼

Fig for working flow

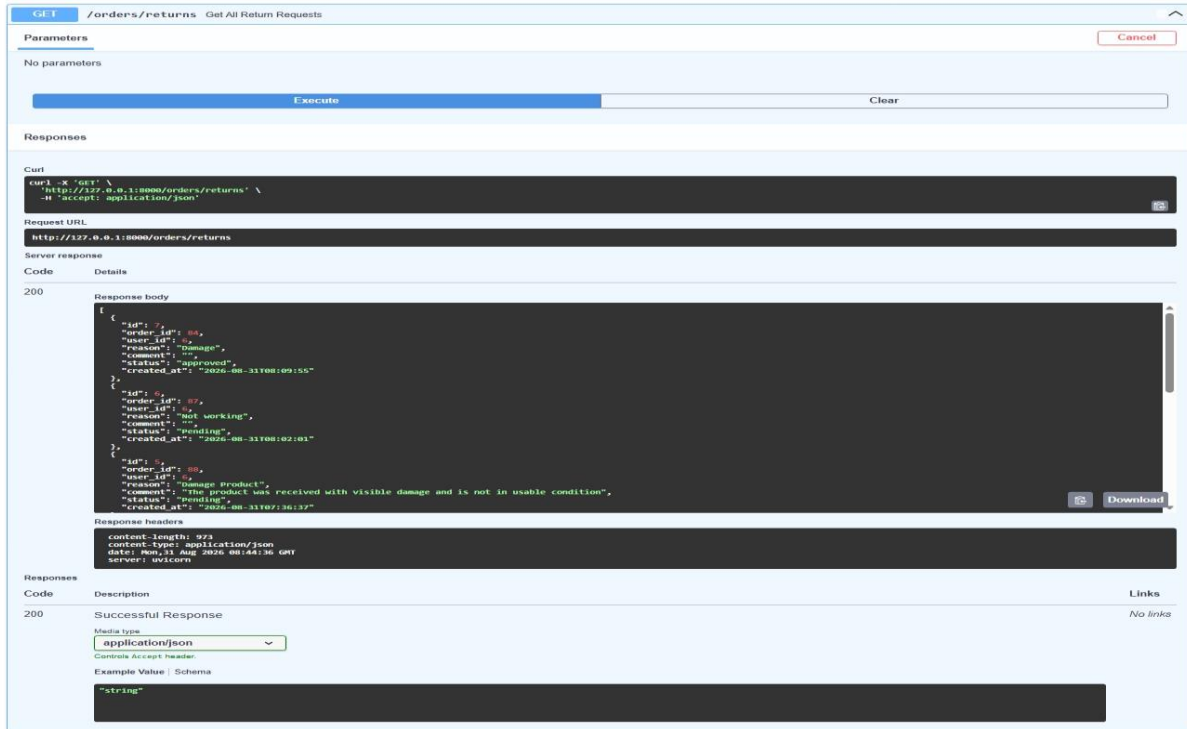


Fig For All Return Request

4. RETURN REQUEST FORM

The User Orders page provides a return request interface.

The form contains:

- **Reason** – Required field.
- **Comment** – Optional field.
- **Submit Return Request** button.

The submitted information is sent to the FastAPI backend.

5. API IMPLEMENTATION & RETURN VALID

The following API has been implemented:

POST /orders/{order_id}/return

This API creates a new return request for an eligible order.

Request Data

```
{  
  "reason": "Product damaged",  
  "comment": "The product was received in damaged condition."  
}
```

The system first checks whether the requested order exists. If the order does not exist, an appropriate error response is returned.

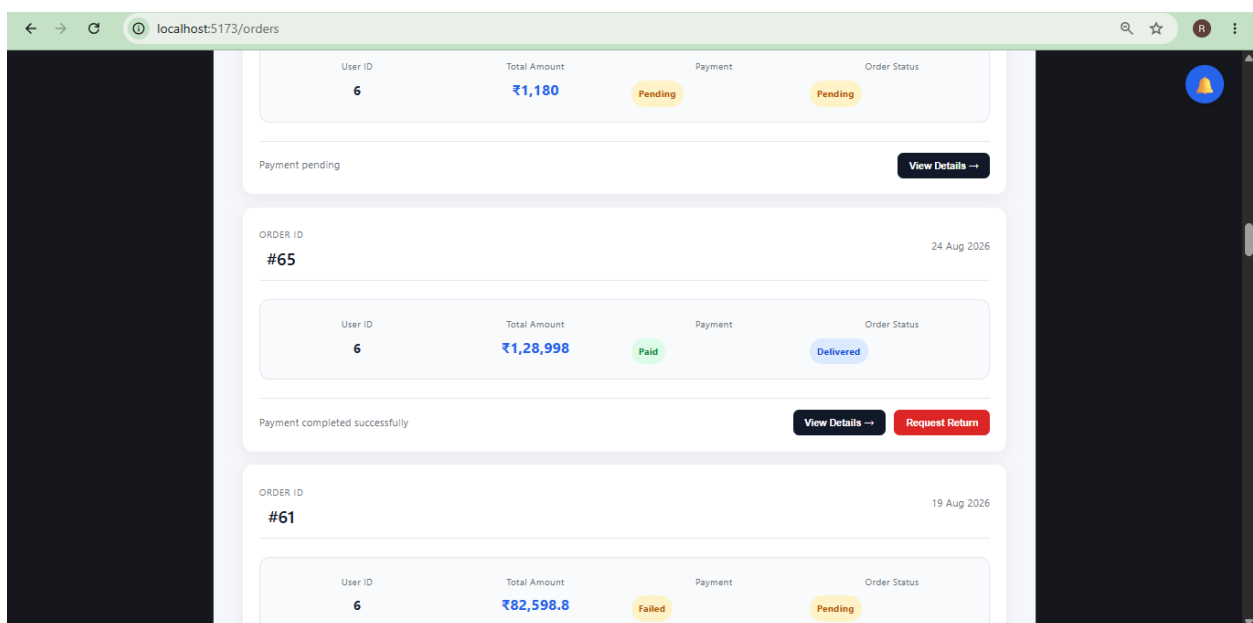


Fig for Frontend Return Request

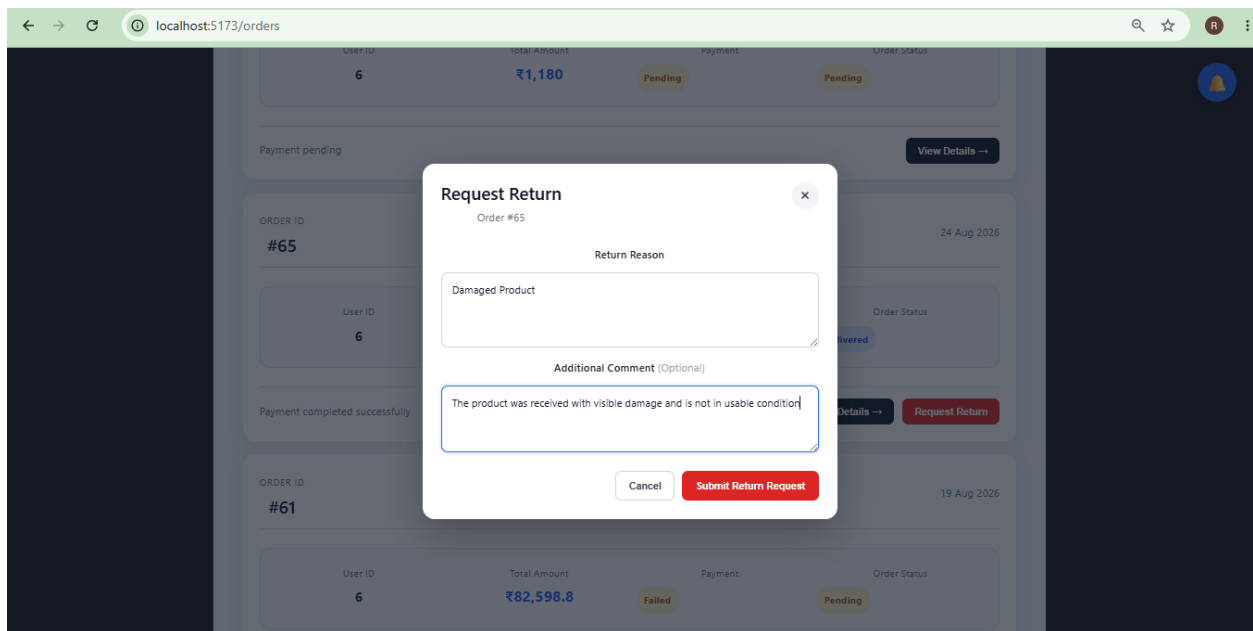
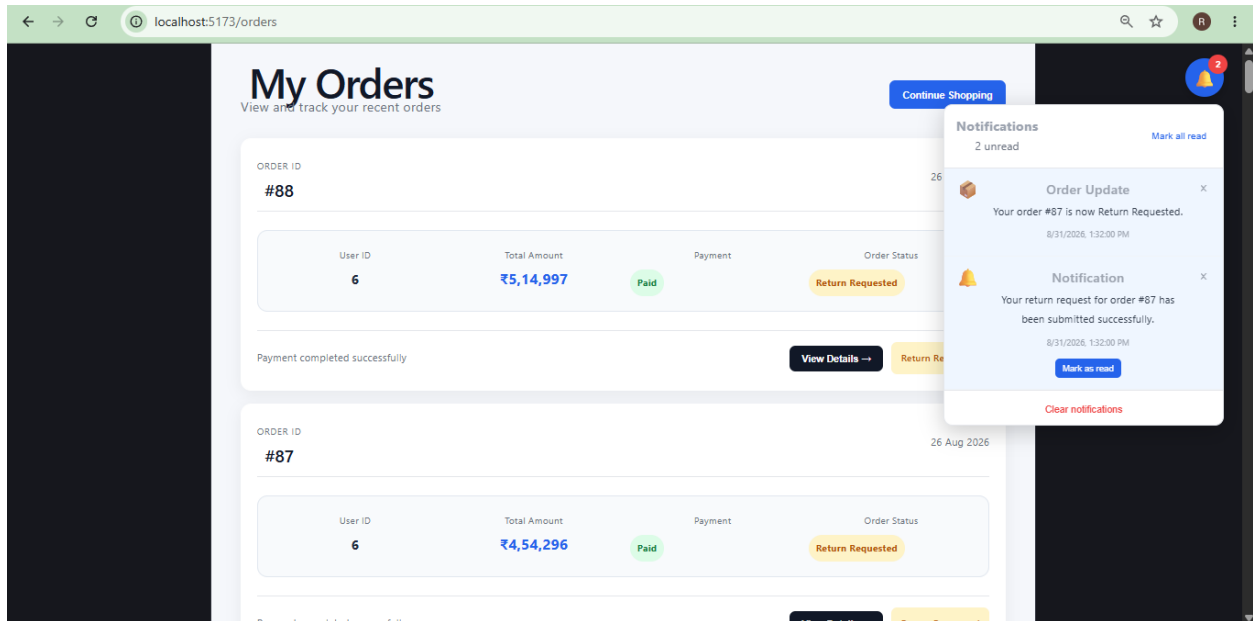


Fig for Frontend Return Request raising by reason and command

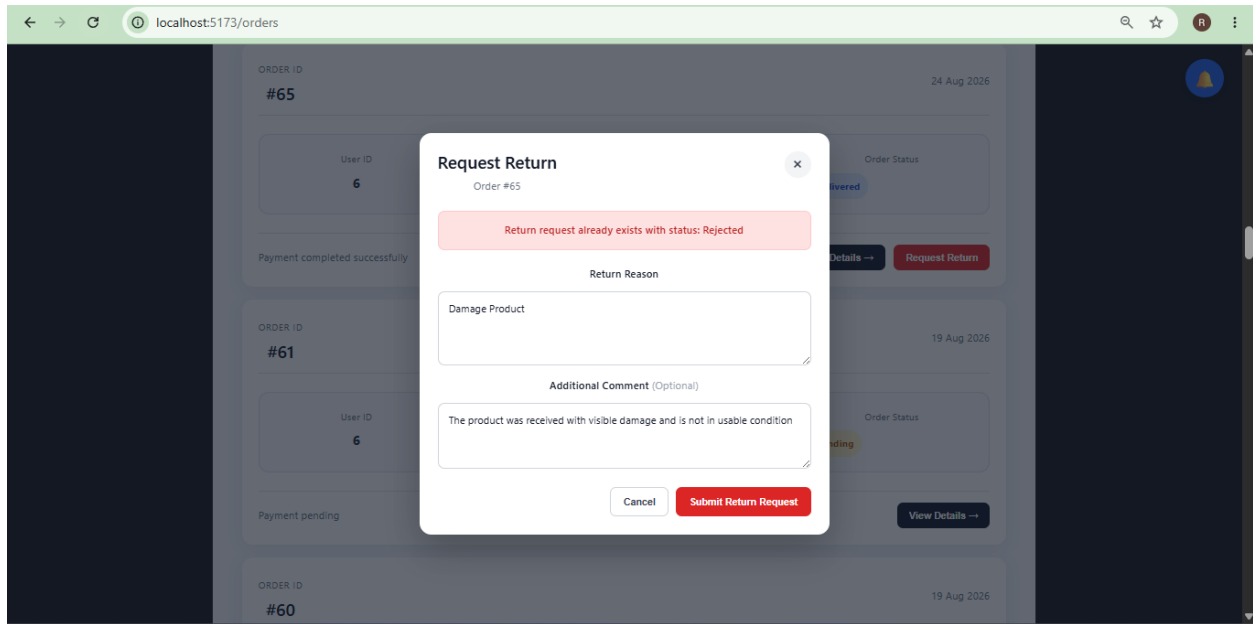
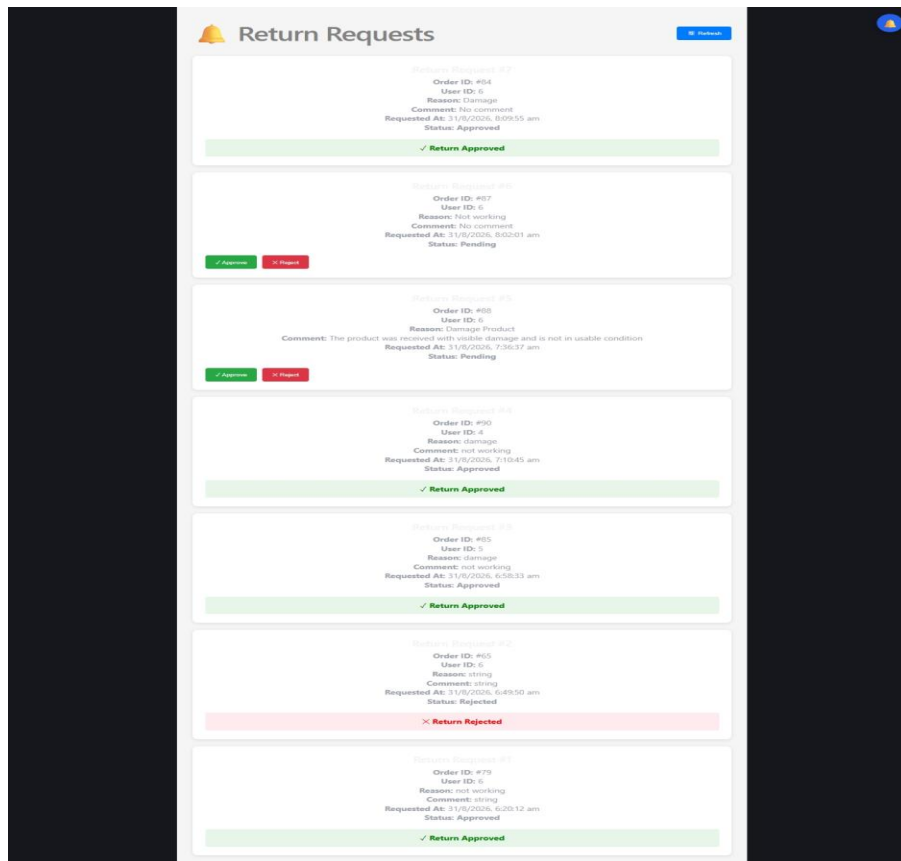


Fig For Rejected



For Admin Approve, Reject And Pending Frontend

6. DATABASE DESIGN

A dedicated **ReturnRequest** model has been implemented to store return information.

The screenshot shows the MySQL Workbench interface. The left sidebar contains the 'MANAGEMENT' and 'INSTANCE' sections. The main window displays a query window with the following SQL commands:

```
221 ADD COLUMN delivered_at DATETIME NULL;  
222 DESCRIBE orders;  
223 DESCRIBE return_requests;  
224  
225
```

The 'Result Grid' shows the schema for the 'return_requests' table:

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI		auto_increment
order_id	int	NO	MUL		
user_id	int	NO	MUL		
reason	varchar(255)	NO			
comment	text	YES			
status	varchar(30)	NO			
created_at	datetime	NO			

The 'Output' window shows the results of the 'DESCRIBE return_requests' query:

#	Time	Action	Message	Duration / Fetch
51	12:55:19	Describe return_requests	7 row(s) returned	0.000 sec / 0.000 sec
52	12:55:19	Describe return_requests	7 row(s) returned	0.000 sec / 0.000 sec
53	12:55:19	Describe return_requests	7 row(s) returned	0.016 sec / 0.000 sec
54	12:55:19	Describe return_requests	7 row(s) returned	0.000 sec / 0.000 sec
55	12:55:20	Describe return_requests	7 row(s) returned	0.000 sec / 0.000 sec
56	12:55:20	Describe return_requests	7 row(s) returned	0.000 sec / 0.000 sec

The screenshot shows the MySQL Workbench interface. The left sidebar contains the 'MANAGEMENT' and 'INSTANCE' sections. The main window displays a query window with the following SQL commands:

```
219 DESCRIBE orders;  
220 ALTER TABLE orders  
221 ADD COLUMN delivered_at DATETIME NULL;  
222 DESCRIBE orders;  
223 select * from return_requests;
```

The 'Result Grid' shows the results of the 'select * from return_requests' query:

#	Time	Action	Message	Duration / Fetch
55	12:55:20	Describe return_requests	7 row(s) returned	0.000 sec / 0.000 sec
56	12:55:20	Describe return_requests	7 row(s) returned	0.000 sec / 0.000 sec
57	12:56:16	select * from return_requests LIMIT 0, 1000	4 row(s) returned	0.000 sec / 0.000 sec
58	13:09:16	select * from return_requests LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
59	13:31:42	select * from return_requests LIMIT 0, 1000	5 row(s) returned	0.000 sec / 0.000 sec
60	14:10:13	select * from return_requests LIMIT 0, 1000	7 row(s) returned	0.000 sec / 0.000 sec

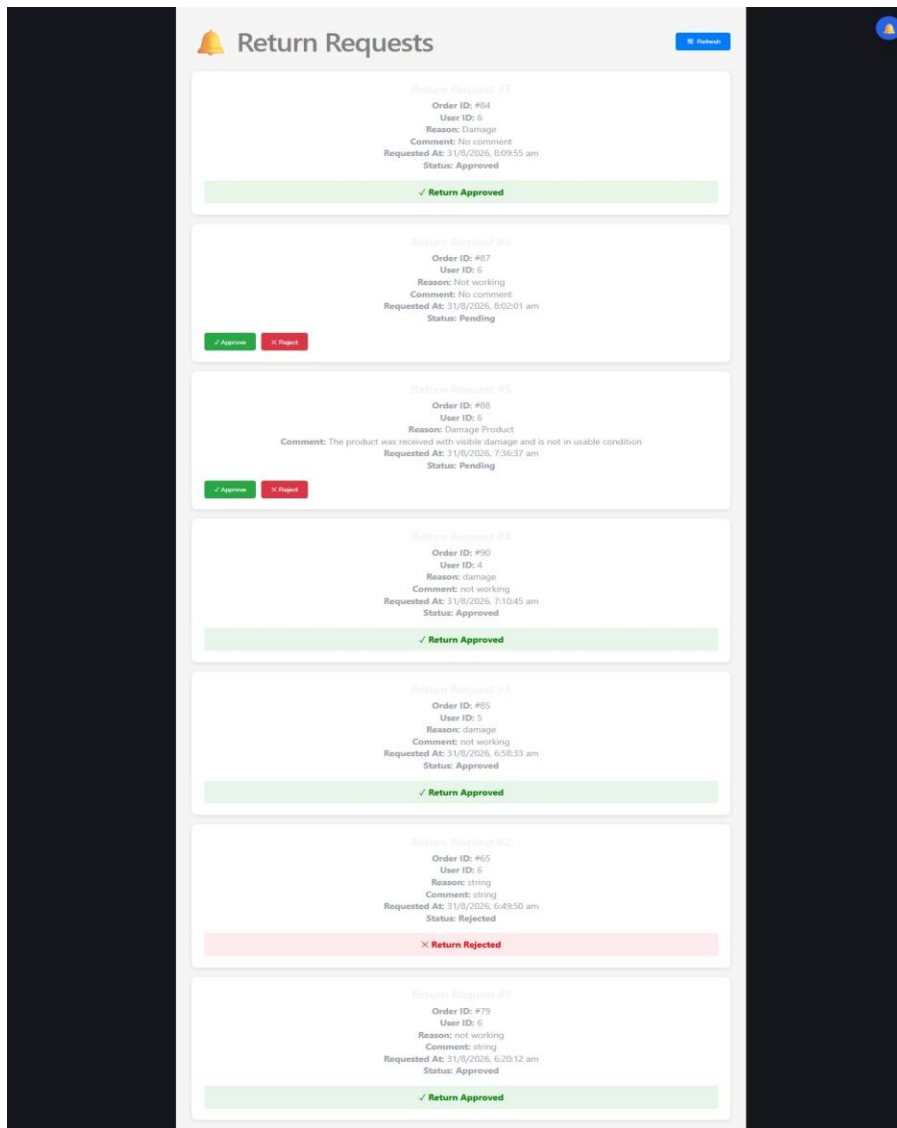
Database model

7. ADMIN RETURN MANAGEMENT

An Admin Return Requests page has been implemented to manage customer return requests.

The administrator can:

- View all return requests.
- View current return status.
- Approve pending requests.
- Reject pending requests.



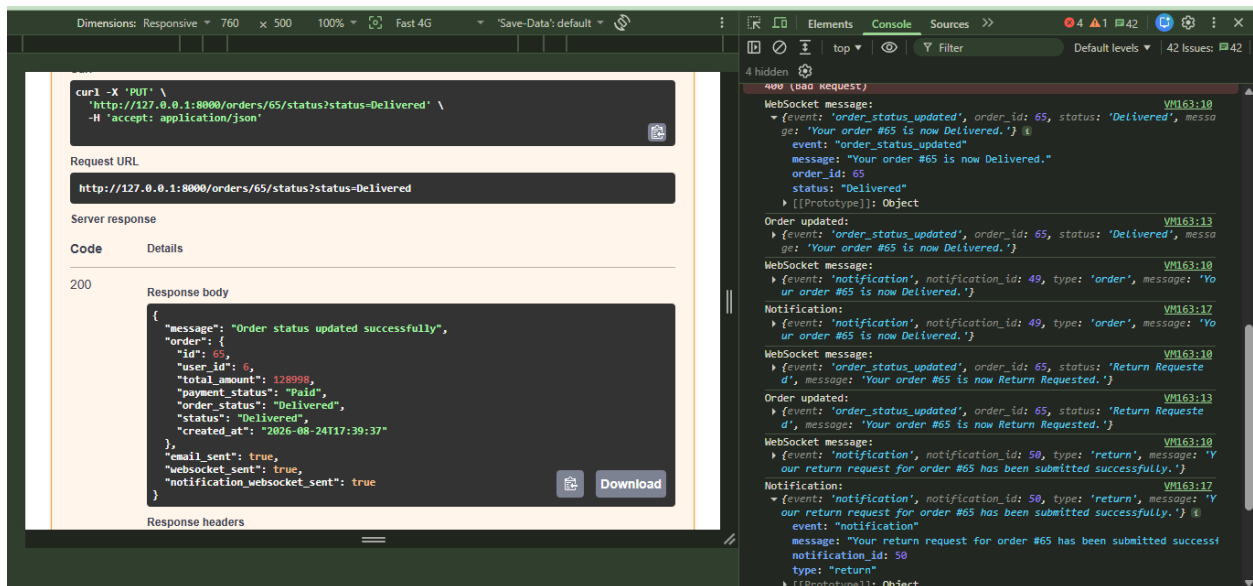
8. REAL-TIME WEBSOCKET UPDATES

WebSocket functionality has also been integrated with the return workflow.

Real-time updates are generated when:

- A return request is submitted.
- A return request is approved.
- A return request is rejected.

This improves the real-time customer experience.



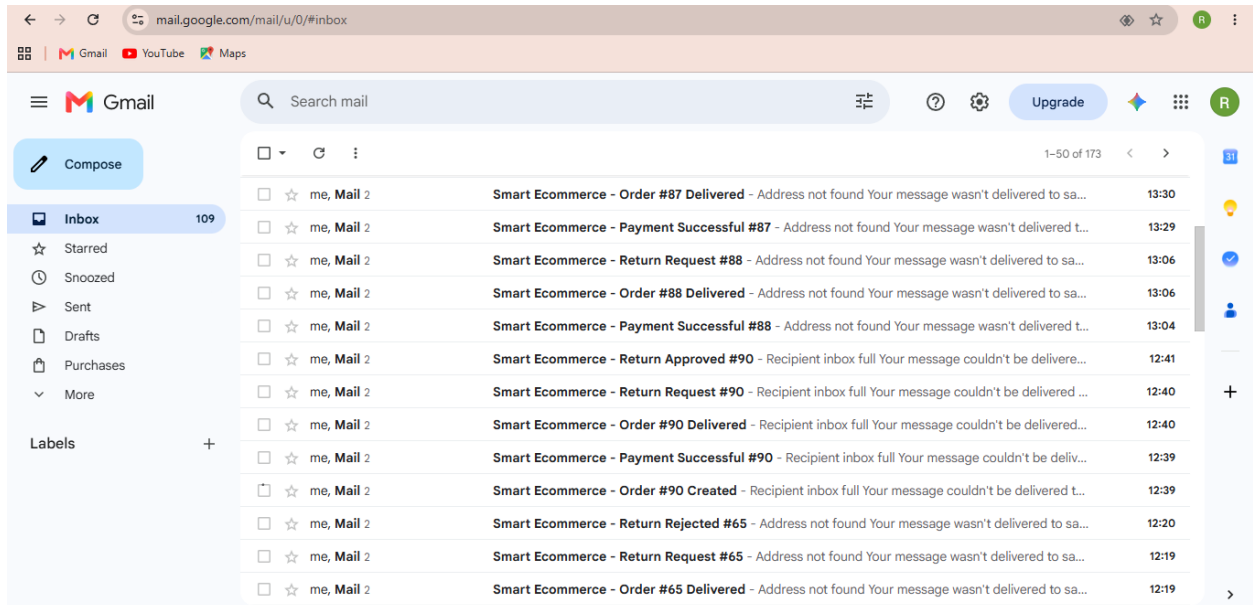
9. EMAIL NOTIFICATIONS

Email notifications are integrated into the return workflow.

Emails can be sent when:

- A return request is submitted.
- A return request is approved.
- A return request is rejected.

The email system is implemented with error handling so that an email failure does not stop the main return operation.



10. ADMIN RETURN REQUEST SCREEN

The Admin Return Requests screen displays all customer return requests.

Example information displayed:

Return Request #7

Order ID: #84

User ID: 6

Reason: Damage

Comment: No comment

Status: approved

11. API RESPONSE

After successfully submitting a return request, the API returns information such as:

```
{
  "message": "Return request submitted successfully",
  "return_request": {
    "id": 7,
    "order_id": 84,
    "user_id": 6,
    "reason": "Damage",
    "comment": "Product was damaged",
    "status": "pending"
  },
  "order_status": "Return Requested"
}
```

The response confirms that the return request was successfully created.

12. ERROR HANDLING

The return module handles invalid operations using FastAPI HTTP exceptions.

Examples include:

- Order not found.
- Order is not delivered.
- Return window expired.
- Return request already exists.
- Invalid return request status.
- Return request not found.

Appropriate HTTP status codes and error messages are returned to the client.

13. TESTING

The complete Return Request workflow was tested using the application frontend and backend APIs.

The following scenarios were verified:

Successful Request

Delivered Order
↓
Request Return
↓
Return Request Created
↓
Status = pending
↓
Order = Return Requested

Approved Request

Pending
↓
Admin Approves
↓
approved
↓
Order = Returned

Rejected Request

Pending
↓
Admin Rejects => rejected -> Order = Delivered

14. IMPLEMENTED DELIVERABLES

The assigned deliverables have been completed:

- ☒ Return Request API
- ☒ User Orders UI integration
- ☒ Request Return functionality
- ☒ ReturnRequest database model
- ☒ Reason and optional comment
- ☒ 7-day return validation
- ☒ Delivered-order validation
- ☒ Duplicate return validation
- ☒ Order status update
- ☒ Admin Return Requests page
- ☒ Approve/Reject functionality
- ☒ Database notifications
- ☒ WebSocket updates
- ☒ Email notifications
- ☒ Error handling
- ☒ End-to-end testing

15. CONCLUSION

The **User-side Refund & Return Request Flow** has been successfully implemented as part of the Customer Experience & Insights Module. The implementation provides a structured return workflow from customer request to administrative processing. It includes return eligibility validation, database storage, order status management, admin approval/rejection, notifications, real-time updates, and email communication. The requested assignment deliverables have been completed and the complete return request workflow has been tested successfully.

Task Status: COMPLETED ☒